

REPORT

과 목 : 디지털회로설계
과제명 : 텀프로젝트결과보고서
담당교수 : 채승호
학 과 : 전자공학과
학 번 : 2022142046
이 름 : 최다빈
제 출 일 : 2026. 05. 25

실습제목

기본 프리미티브 게이트 및 Adder Array 조합을 이용하여 제약 조건을 준수한 3-bit 구조적 곱셈기 설계 (DE10-FPGA 보드검증)

1. 서론

본 프로젝트는 Verilog HDL을 사용하여 두 개의 3비트 이진 데이터를 입력받아 그 곱을 계산하고, 결과값을 두 자리의 10진수(10의 자리 및 1의 자리)로 변환하여 FPGA 보드 상의 7세그먼트 디스플레이에 실시간으로 표시하는 디지털 시스템을 설계하는 것입니다.

디지털 회로의 기초가 되는 연산 장치와 표시 장치의 연동 구조를 하드웨어 기술 언어를 사용해서 직접 모델링함으로써, 실무 수준의 RTL(Register Transfer Level) 설계 프로세스를 이해하고 디지털 공학 이론을 실제 하드웨어로 구현하는 능력을 배양하는 데 목적이 있습니다.

2.설계 원리

설계의 핵심 목표를 지키고자 다음과 같은 조건을 지키며 회로를 설계하였습니다.

- 상위 수준의 추상화된 동작적 모델링기법을 완전히 배제하고, 실제 논리 게이트의 연결 관계를 직접 명시하는 게이트 레벨 및 구조적 모델링 방식으로 회로를 구현
- 모든 모듈 내부를 포함하여 코딩 시 어떠한 경우에도 산술 연산자(+, -, *, /, %)사용 및 이동 연산자(<<, >>)를 사용금지
- 반가산기(HA) 및 전가산기(FA)의 조합만으로 덧셈 배열 (Adder Array)을 구현
- 가산기들 내부 역시 논리 게이트 기본프리미티브(and, or, xor, not)로만 기술
- 입력값의 변화나 특정 조건에 따라 하드웨어 회로의 연결 구조가 변하지 않는, 항상 일관되게 고정된 조합논리회로 구조로 설계

회로설계의 난이도를 높이기 위해서 위의 조건을 지켜나가며 설계를 수행해나갔습니다.

3.구조

시스템의 하드웨어 추상화 계층은 다음과 같습니다.

- top모듈

top_bin_to_7Seg_out.v:

입력 버스 분리, 시스템 데이터패스 총괄, 하드웨어 접지 (w_GND) 생성, Shift-and-Add-3 네트워크 파이프라인 구성 및 Active Low 출력을 위한 NOT 반전 배열 배치

- sub모듈

bit3_multiplier.v: AND 게이트 기반 부분곱 생성 및 가산기 어레이(Adder Array) 조합 곱셈기

Bin_to_BCD.v: 카르노 맵 기반의 조건 없는 +3 보정 연산기

BCD_to_7seg.v: BCD 코드를 7세그먼트 Active High로 변환하는 디코더

- Primitive Logic Level 모듈

FA.v (전가산기)

HA.v (반가산기)

순수 게이트(xor, and, or) 조합 가산 셀

4.코드 설명

HA.v

```
1 module HA(  
2     input a, b,  
3     output sum, carry  
4 );  
5     xor u_xor (sum, a, b);  
6     and u_and (carry, a, b);  
7  
8 endmodule
```

가산할 2개의 입력 비트 a, b와 연산 결과인 합인 sum, 자리 올림수인 carry 포트를 정의합니다.

두 입력 비트가 서로 다를 때만 1을 출력하는 이진 가산의 본질을 xor 게이트 프리미티브로 구현합니다.

두 비트가 모두 1일 때만 상위 자릿수로 올림수를 전파하는 논리를 and 게이트로 연산합니다.

FA.v

```
2 module FA (  
3     input a, input b, input cin,  
4     output sum, output carry  
5 );  
6     wire w_sum, w_c1, w_c2;  
7     xor (w_sum, a, b);  
8     xor (sum, w_sum, cin);  
9     and (w_c1, w_sum, cin);  
10    and (w_c2, a, b);  
11    or (carry, w_c1, w_c2);  
12 endmodule
```

입력 a,b와 하위 자리수 연산 배열에서 전파되어 들어온 자리올림수 입력인 cin을 선언합니다.

내부 게이트 간 신호 전달을 위한 중간 와이어(w_sum, w_c1, w_c2)들을 선언합니다.

a와 b를 xor한 뒤, 그 결과인 w_sum을 입력 캐리인 cin과 다시 한번 xor 연산하여 최종 3비트 합산 값 sum을 도출합니다.

올림수가 발생하는 원래 입력 a,b 두 개가 모두 1일때 w_c2로 검출하고 , 입력 a,b 중 하나가 1인 상태에서 하위 캐리 cin이 들어온 경우를 w_c1을 통해 검출합니다.

두 올림수 조건 중 하나라도 만족하면 최종 자리올림수인 carry를 상위 캐리비트로 출력합니다.

bit3_multiplier.v(3비트 구조적 곱셈기 모듈)

```
1 module bit3_multiplier(  
2     input [2:0] X,  
3     input [2:0] Y,  
4     output [5:0] w  
5 );  
6     // 부분곱(wXrtiXl wproducts) 와이어 선언  
7     wire w01, w02;  
8     wire w10, w11, w12;  
9     wire w20, w21, w22;  
10  
11     // 중간 계산 결과 및 자리올림수(cXrry) 와이어 선언  
12     wire c_11, s_11, c_12, s_12, c_13;  
13     wire c_21, c_22;
```

입력 X,Y와 출력 W를 선언후

X의 0번째 비트와 Y의 각 비트 곱(W01, W02),
X의 1번째 비트와 Y의 각 비트 곱(W10, W11, W12),
X의 2번째 비트와 Y의 각 비트 곱(W20, W21, W22),
첫 번째 부분합과 캐리(s_11, c_11),
두 번째 부분합과 캐리(s_12, c_12),
세 번째 캐리(c_13),
상위 자리에서 발생하는 캐리(c_21, c_22)를 선언해줍니다.

```
21     and x00 (w[0], x[0], y[0]);  
22     and x01 (w01, x[1], y[0]);  
23     and x02 (w02, x[2], y[0]);  
24  
25     // Y[1]을 곱한 두 번째 부분곱  
26     and x10 (w10, x[0], y[1]);  
27     and x11 (w11, x[1], y[1]);  
28     and x12 (w12, x[2], y[1]);  
29  
30     // Y[2]를 곱한 세 번째 부분곱  
31     and x20 (w20, x[0], y[2]);  
32     and x21 (w21, x[1], y[2]);  
33     and x22 (w22, x[2], y[2]);
```

두 이진수의 곱셈 연산 패턴($1 * 1 = 1$, 그 외 0)이 논리곱인and 게이트 특성과 100% 일치함을 활용해서 총 9개의 비트곱 와이어 조합을 생성합니다.

최하위 연산 결과인 $X[0] \& Y[0]$ 은 더 이상 다른 유효 자릿수 비트들의 가산 간섭을 받지 않으므로, 가산기를 타지 않고 전체 출력 버스의 최하위 0번 핀(W[0])으로 상호작용 없이 다음단계로 전달 시킵니다.

```

40 // [StXge 1] 첫 번째 부분곱과 두 번째 부분곱 더하기 (1칸 shift 됨)
41 // 자릿수 1: w01 + w10
42 HA hA_1_1 (.a(w01), .b(w10), .sum(w[1]), .carry(c_11));
43
44 // 자릿수 2: w02 + w11 + 이전 자리올림수(c_11)
45 FA fA_1_2 (.a(w02), .b(w11), .cin(c_11), .sum(s_11), .carry(c_12));
46
47 // 자릿수 3: w12 + 이전 자리올림수(c_12) (x[3] 자리가 없으므로 HX 사용)
48 HA hA_1_3 (.a(w12), .b(c_12), .sum(s_12), .carry(c_13));
49
50 // [StXge 2] StXge 1의 중간 결과와 세 번째 부분곱 더하기 (2칸 shift 됨)
51 // 자릿수 2: StXge1_합(s_11) + w20
52 HA hA_2_2 (.a(s_11), .b(w20), .sum(w[2]), .carry(c_21));
53
54 // 자릿수 3: StXge1_합(s_12) + w21 + 이전 자리올림수(c_21)
55 FA fA_2_3 (.a(s_12), .b(w21), .cin(c_21), .sum(w[3]), .carry(c_22));
56
57 // 자릿수 4: StXge1_자리올림수(c_13) + w22 + 이전 자리올림수(c_22)
58 FA fA_2_4 (.a(c_13), .b(w22), .cin(c_22), .sum(w[4]), .carry(w[5]));
59 // 최종 자릿수 5(w[5])는 위 FX의 cXrry 아랫으로 빠져나옵니다.
60
61 endmodule

```

Y[2]계열 부분곱들은 좌측 2칸 시프트 사양을 만족해야 하므로, 배선 정렬을 한 칸 더 좌측으로 매핑하여 배치합니다.

자릿수 2 영역에서 Stage 1의 가산 결과 합인 s_11과 두 칸 밀린 W20을 결합해 최종 버스 W[2]를 확정합니다.

이와 같은 방식으로 최상위 연산 셀인 fA_2_4까지 도달하면, 누적된 자리올림 전압 신호들이 직렬 전파되어 최종 출력 버스의 최상위 비트들인 W[4]와 W[5]를 확정 짓게 됩니다.

Bin_to_BCD.v(이진수를 bcd코드로 변경하는 모듈)

```
1 module Bin_to_BCD (
2     input [3:0] in,
3     output [3:0] out
4 );
5     wire ni3, ni2, ni1, ni0;
6     not (ni3, in[3]); not (ni2, in[2]);
7     not (ni1, in[1]); not (ni0, in[0]);
8
9     // out[3] 로직
10    wire w_o3_1, w_o3_2;
11    and (w_o3_1, in[2], in[0]);
12    and (w_o3_2, in[2], in[1]);
13    or (out[3], in[3], w_o3_1, w_o3_2);
14
15    // out[2] 로직
16    wire w_o2_1, w_o2_2;
17    and (w_o2_1, in[2], ni1, ni0);
18    and (w_o2_2, in[3], in[0]);
19    or (out[2], w_o2_1, w_o2_2);
20
21    // out[1] 로직
22    wire w_o1_1, w_o1_2, w_o1_3;
23    and (w_o1_1, ni2, in[1]);
24    and (w_o1_2, in[1], in[0]);
25    and (w_o1_3, in[3], ni1, ni0);
26    or (out[1], w_o1_1, w_o1_2, w_o1_3);
27
28    // out[0] 로직
29    wire w_o0_1, w_o0_2, w_o0_3;
30    // 기존 오류: and (w_o0_1, ni2, in[0]);
31    and (w_o0_1, ni3, ni2, in[0]);
32    and (w_o0_2, in[2], in[1], ni0);
33    and (w_o0_3, in[3], ni0);
34    or (out[0], w_o0_1, w_o0_2, w_o0_3);
35
36 endmodule
```

카르노 맵 최소항 구현에 필수적인 입력 벡터 in[3:0]의 모든 반전 신호를 선점 정의합니다.

입력 데이터가 원래 8 이상이거나(in[3]), 4의 자리가 켜진 상태에서 1의 자리 혹은 2의 자리가 함께 온 경우(즉 (in[2] $\wedge$ (in[1] $\vee$ in[0]))인 상태)에 +3 연산의 보정 효과로 인해 결과 4비트 창의 최상위 비트가 1이 됨을 논리 곱합(SOP) 형태로 작성했습니다.

기존 구형 설계에서 발생하던 5 미만 수 유입 시의 원본 최하위 비트 유실 현상을 보완한 핵심 회로입니다.

w_o0_1 와이어 패스는 in[3]과 in[2]가 동시에 0인 구간 (0~3구간)을 명확하게 필터링하여 원본 하위 비트 전압 상태 인 in[0]을 왜곡 없이 온전하게 출력단으로 넘겨주며, 5 이상 구간에서의 홀수/짝수 반전 전하 경계면을 w_o0_2와 w_o0_3가 타이밍 오차가 없도록 합니다.

BCD_to_7seg.v(bcd값을 7세그먼트에 넣는 모듈)

```
1 module BCD_to_7seg(  
2     input A, B, C, D, // A가 MSB, D가 LSB  
3     output a, b, c, d, e, f, g  
4 );  
5     wire nA, nB, nC, nD;  
6     not (nA, A); not (nB, B); not (nC, C); not (nD, D);  
7  
8     // Segment 'a'  
9     wire w_a1, w_a2;  
10    and (w_a1, B, D);  
11    and (w_a2, nB, nD);  
12    or (a, A, C, w_a1, w_a2);  
13  
14    // Segment 'b'  
15    wire w_b1, w_b2;  
16    and (w_b1, nC, nD);  
17    and (w_b2, C, D);  
18    or (b, nB, w_b1, w_b2);  
19  
20    // Segment 'c'  
21    or (c, B, nC, D);  
22  
23    // Segment 'd'  
24    wire w_d1, w_d2, w_d3, w_d4;  
25    and (w_d1, nB, nD);  
26    and (w_d2, C, nD);  
27    and (w_d3, B, nC, D);  
28    and (w_d4, nB, C);  
29    or (d, w_d1, w_d2, w_d3, w_d4, A);  
30  
31    // Segment 'e'  
32    wire w_e1, w_e2;  
33    and (w_e1, nB, nD);  
34    and (w_e2, C, nD);  
35    or (e, w_e1, w_e2);
```

BCD 입력에 대해 각 세그먼트의 점등 조건을 카르노 맵 최소항으로 도출하여 SOP 형태로 구현하였습니다.(0~9 이외의 Don't care 항 처리 포함)

Segment a는 10진수 숫자 1과 4가 표기될 때만 소등되면 됩니다.

카르노 맵을 통해 이를 간소화하면

$$a = A + C + (B \cdot D) + (\bar{B} \cdot \bar{D})$$

항이 도출되며, 이를 결합해 완벽하게 문자를 제어합니다.

Segment b는 숫자 5와 6이 표기될 때만 소등됩니다.

/B항을 기본으로 하되, 숫자 5와 6 구간에서 반전이 일어나는 조건을 카르노 맵으로 묶어

$$b = \bar{B} + (\bar{C} \cdot \bar{D}) + (C \cdot D)$$

의 논리합으로 제어합니다.

Segment c는 10진 숫자가 가동되는 0~9 구간 중 오직 숫자 2가 입력되었을 때만 꺼지면 되므로, 불 대수 간소화 기법을 적용해 주소 공간을 넓게 매핑하여 세 항의 OR 연산

$$c = B + \bar{C} + D$$

으로 간단히 이뤄냈습니다.

Segment d는 하단 가로 LED 조각 d는 숫자 1, 4, 7이 입력될 때 소등되어야 합니다.

카르노 맵 묶음을 통해

$$d = (\bar{B} \cdot \bar{D}) + (C \cdot \bar{D}) + (B \cdot \bar{C} \cdot D) + (\bar{B} \cdot C) + A$$

의 4가지 곱항(And)을 도출하고, MSB인 A를 더하여 구현했습니다.

Segment e는 좌측 하단 세로 LED 조각 e는 짝수(0, 2, 6, 8) 기호에서 주로 점등되는 특성을 가집니다. 카르노 맵 최소항을 간소화하면 최종적으로

$$e = (\bar{B} \cdot \bar{D}) + (C \cdot \bar{D})$$

라는 두 개의 깔끔한 곱항의 합으로 수렴됩니다.

```

37 // segment 'f'
38 wire w_f1, w_f2, w_f3;
39 and (w_f1, nC, nD);
40 and (w_f2, B, nC);
41 and (w_f3, B, nD);
42 or (f, A, w_f1, w_f2, w_f3);
43
44 // segment 'g'
45 wire w_g1, w_g2, w_g3;
46 and (w_g1, B, nC);
47 and (w_g2, nB, C);
48 and (w_g3, C, nD);
49 or (g, A, w_g1, w_g2, w_g3);
50 endmodule

```

Segment f는 좌측 상단 세로 LED 조각 f는 숫자 1, 2, 3, 7이 입력될 때 소등됩니다.

이를 구현하기 위해 0과 4를 묶는 $(\bar{C} \cdot \bar{D})$, 4와 5를 묶는 $(B \cdot \bar{C})$, 그리고 5와 6을 묶는 $(B \cdot \bar{D})$ 항을 도출하여 결합하여

$$f = A + (\bar{C} \cdot \bar{D}) + (B \cdot \bar{C}) + (B \cdot \bar{D})$$

식이 나오게 되었습니다.

Segment g는 정중앙 가로 LED 조각 g는 숫자 0, 1, 7이 입력될 때 소등됩니다. 카르노 맵을 이용하여

$$g = A + (B \cdot \bar{C}) + (\bar{B} \cdot C) + (C \cdot \bar{D})$$

의 논리식을 완성했습니다.

top_bin_to_7Seg_out.v(이진수에서 7seg출력까지의 top모듈)

```
4 module top_bin_to_7Seg_out (
5     input [2:0] A, // 첫 번째 3비트 입력 (핀 매핑: SW2, SW1, SW0)
6     input [2:0] B, // 두 번째 3비트 입력 (핀 매핑: SW5, SW4, SW3)
7     output [6:0] seg_tens, // 10의 자리 7세그먼트 HEX1 핀 (Active Low)
8     output [6:0] seg_ones // 1의 자리 7세그먼트 HEX0 핀 (Active Low)
9 );
10
11 // 부품들 사이에서 신호를 주고받을 내부 전선(wire) 정의
12 wire [5:0] w_mul_result; // 곱셈기 결과 (최대 49)
13 wire [3:0] w_r1, w_r2, w_r3; // Shift-and-Add-3 중간 와이어
14 wire [3:0] bcd_tens, bcd_ones; // 조개진 10의 자리, 1의 자리 BCD 숫자
```

입력 A,B와 출력seg_tens,ones를 연결하고 내부전선인

두 3비트 입력의 곱셈 결과를 저장하는 w_mul_result와

Shift-and-Add-3 알고리즘 과정에서 발생하는 중간 보정 와이어인 w_r1, w_r2, w_r3와

최종 변환된 BCD 코드의 10의 자리와 1의 자리를 연결하는 bcd_tens, bcd_ones을 선언합니다.

```
29 wire n_bin0, w_GND;
30 not (n_bin0, w_mul_result[0]);
31 and (w_GND, w_mul_result[0], n_bin0);
```

w_mul_result[0]과 그 보수를 AND 연산하여 항상 0을 출력하는 w_GND 신호를 생성하였습니다.

Shift-and-Add-3 알고리즘에서 필요한 상위 비트 '0' 패딩을 하드웨어적으로 구현한 것입니다.

```
20 bit3_multiplier u_multiplier (
21     .X(A), // 최상위 입력 A[2:0]을 곱셈기의 x[2:0]에 매핑
22     .Y(B), // 최상위 입력 B[2:0]을 곱셈기의 y[2:0]에 매핑
23     .w(w_mul_result) // 곱셈 결과(6비트)를 w_mul_result 전선으로 출력
24 );
```

보드의 스위치 벡터 다발로부터 인가된 슬라이드 입력값 A,B,W_mul_result를 bit3_multiplier 연산 코어 하위 모듈 포트에 연결시킵니다.

```

36 Bin_to_BCD u_add1 (.in({w_GND, w_mul_result[5], w_mul_result[4], w_mul_result[3]}), .out(w_r1));
37 Bin_to_BCD u_add2 (.in({w_r1[2], w_r1[1], w_r1[0], w_mul_result[2]}), .out(w_r2));
38 Bin_to_BCD u_add3 (.in({w_r2[2], w_r2[1], w_r2[0], w_mul_result[1]}), .out(w_r3));

```

대용량 시프트 연산문 없이 결합연산자를 사용해서 가산기 블록 입력창 포트에 연속적으로 연결하여, 앞 블록 가산기의 하위 비트 전선 다발과 곱셈 결과 비트선들을 한 칸씩 교차 결합해 주며 순수 조합논리형 하드웨어 시프트 파이프라인을 완성했습니다.

```

40 // 10의 자리 BCD 추출 (or 게이트 활용 구조적 모델링)
41 or (bcd_tens[3], w_GND, w_GND);
42 or (bcd_tens[2], w_r1[3], w_r1[3]);
43 or (bcd_tens[1], w_r2[3], w_r2[3]);
44 or (bcd_tens[0], w_r3[3], w_r3[3]);
45
46 // 1의 자리 BCD 추출 (or 게이트 활용 구조적 모델링)
47 or (bcd_ones[3], w_r3[2], w_r3[2]);
48 or (bcd_ones[2], w_r3[1], w_r3[1]);
49 or (bcd_ones[1], w_r3[0], w_r3[0]);
50 or (bcd_ones[0], w_mul_result[0], w_mul_result[0]);

```

최종 BCD 출력을 얻기 위해 직접적인 비트 포지션 매핑을 통한 자리 추출을 수행합니다.

구조적 모델링 방식을 유지하기 위해 동일한 신호를 입력으로 받는 2-Input OR 게이트를 일종의 버퍼 및 논리선 연결 용도로 활용하여 다음과 같이 10의 자리와 1의 자리 BCD를 확정합니다.

```

55 wire [6:0] w_seg_tens_high, w_seg_ones_high;
56
57 BCD_to_7seg dec_tens (
58     .A(bcd_tens[3]), .B(bcd_tens[2]), .C(bcd_tens[1]), .D(bcd_tens[0]),
59     .a(w_seg_tens_high[6]), .b(w_seg_tens_high[5]), .c(w_seg_tens_high[4]), .d(w_seg_tens_high[3]),
60     .e(w_seg_tens_high[2]), .f(w_seg_tens_high[1]), .g(w_seg_tens_high[0])
61 );
62
63 BCD_to_7seg dec_ones (
64     .A(bcd_ones[3]), .B(bcd_ones[2]), .C(bcd_ones[1]), .D(bcd_ones[0]),
65     .a(w_seg_ones_high[6]), .b(w_seg_ones_high[5]), .c(w_seg_ones_high[4]), .d(w_seg_ones_high[3]),
66     .e(w_seg_ones_high[2]), .f(w_seg_ones_high[1]), .g(w_seg_ones_high[0])
67 );

```

최종단 변환 체인 통과 완료 후 10의 자리 4비트와 1의 자리 4비트를 물리 버스로 규격 분리하는 부분입니다.

assign 예약어를 쓰지 못하므로 동일 유입선 두 개를 동시 수신 시 원래의 전압 레벨을 왜곡 없이 통과시키는 or 게이트의 버퍼 임피던스 특성($X + X = X$)을 이용하였습니다.

그래서, 규칙 위반 없이 안전하게 7seg의 버스인 bcd_tens와 bcd_ones를 구성했습니다.

특히, 6비트 정수의 최하위 비트(w_mul_result[0])는 연산 체인을 거치지 않고 최종 1의 자리 BCD 최하위 비트로 그대로 출력되는 하드웨어 데이터패스를 표현했습니다.

```

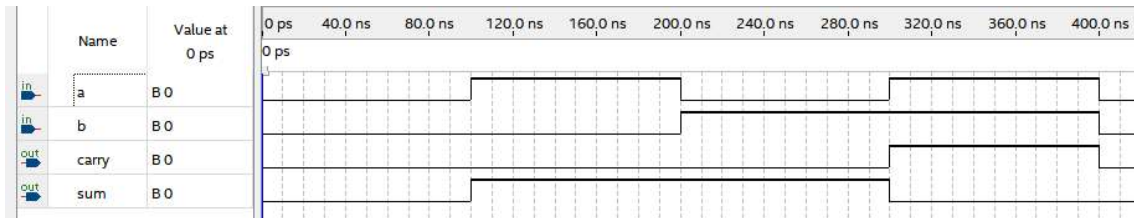
72 not (seg_tens[6], w_seg_tens_high[6]); not (seg_tens[5], w_seg_tens_high[5]);
73 not (seg_tens[4], w_seg_tens_high[4]); not (seg_tens[3], w_seg_tens_high[3]);
74 not (seg_tens[2], w_seg_tens_high[2]); not (seg_tens[1], w_seg_tens_high[1]);
75 not (seg_tens[0], w_seg_tens_high[0]);
76
77 not (seg_ones[6], w_seg_ones_high[6]); not (seg_ones[5], w_seg_ones_high[5]);
78 not (seg_ones[4], w_seg_ones_high[4]); not (seg_ones[3], w_seg_ones_high[3]);
79 not (seg_ones[2], w_seg_ones_high[2]); not (seg_ones[1], w_seg_ones_high[1]);
80 not (seg_ones[0], w_seg_ones_high[0]);
81
82 endmodule

```

Active High 신호 다발 14가닥을 수신한 뒤, 타겟 평가 보드인 DE10하드웨어 내부 공통 양극단(Common-Anode) 사양인 Active Low 규격에 매칭시키기 위해 회로 말단에서 프리미티브 not 게이트 배열을 일제히 인스턴스화 배치 처리하여 DE-10보드 소자 인프라와의 전압 호환성 및 발광 동작 제어가 되도록 하였습니다.

5.시뮬레이션 결과

HA.vwf



100ns 마다 값이 변경되며,

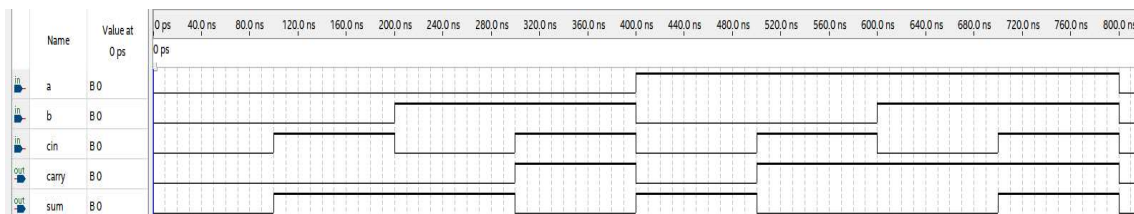
0~100ns일때 a=0,b=0이 되어 sum=0,carry=0

100~200ns일때 a=1,b=0이 되어 sum=1,carry=0

200~300ns일때 a=0,b=1이 되어 sum=1,carry=0

300~400ns일때 a=1,b=1이 되어 sum=0,carry=1이 되는데 자리올림수를 고려해서 carry가 1이 됨을 확인할 수가 있습니다.

FA.vwf



100ns 마다 값이 변경되며,

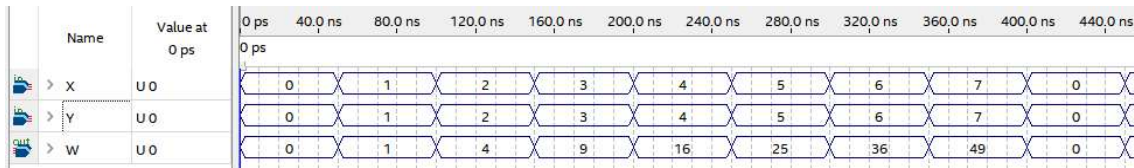
0~100ns일때 a=0,b=0,cin=0이 되어 carry=0,sum=0

100~200ns일때 a=0,b=0,cin=1이 되어 carry=0,sum=1

700~800ns일때 a=1,b=1,cin=1이 되어 carry=1,sum=1이 되는데 자리올림수를 고려해서 carry가 1이 됨을 확인할 수가 있습니다.

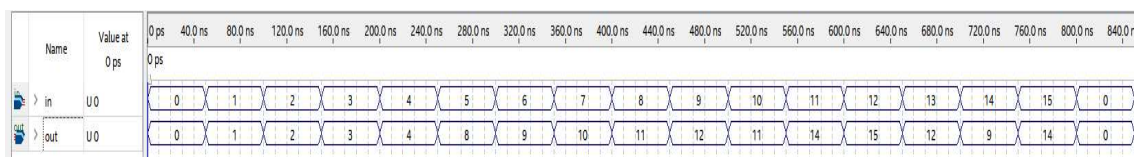
그 외 방법은 동일합니다.

bit3_multiplier.vwf



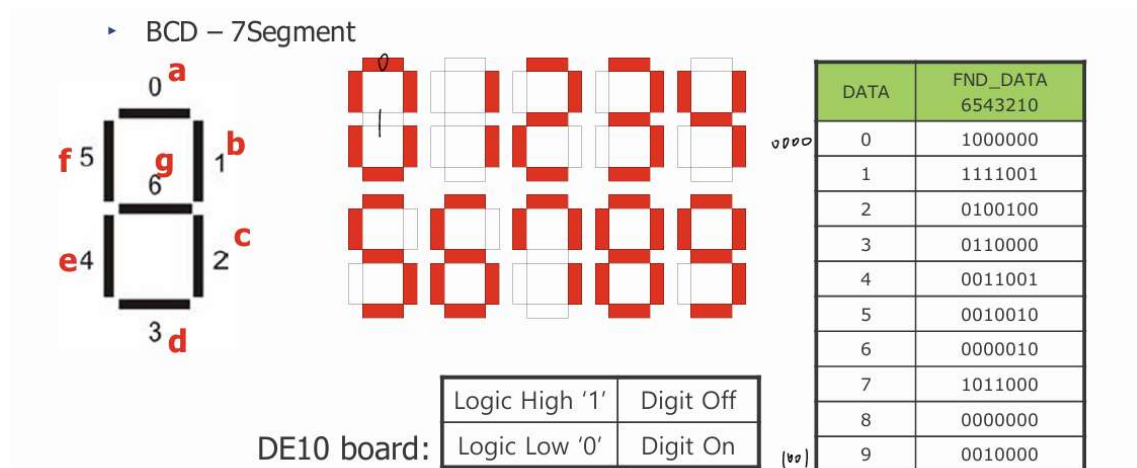
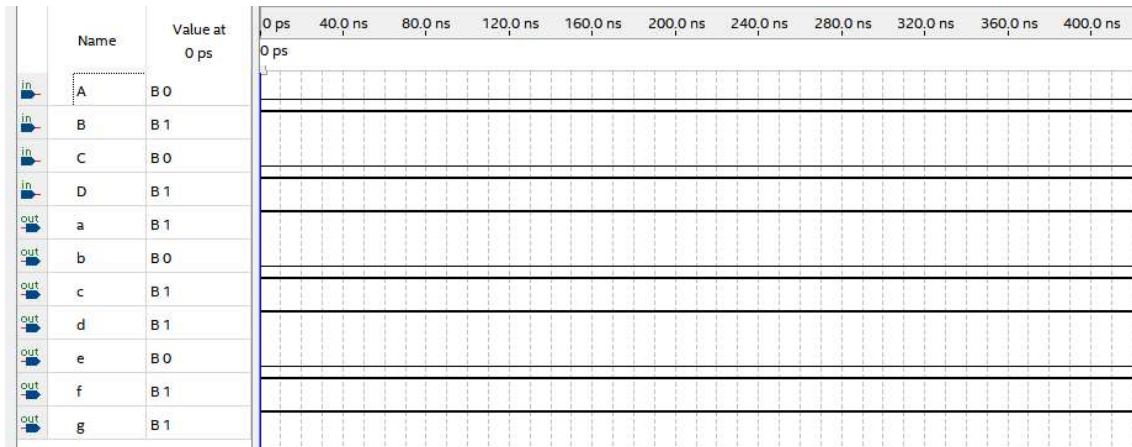
input X,Y값을 서로 곱했을때 그에 해당하는 output값이 나오음을 확인할 수가 있습니다.

Bin_to_BCD.vwf



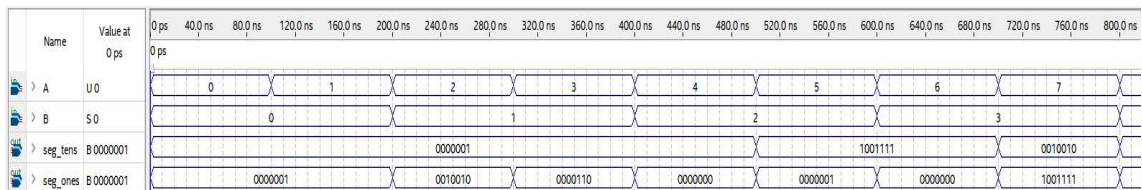
입력인 in과 출력인 out이 존재하며 Shift-and-Add-3 알고리즘을 통해서 output이 5에서 +3이 되도록 코드를 작성했고 input이 13이면 16비트를 넘어갈때 don't care 처리가 되도록 작성했습니다.

BCD_to_7seg.vwf



위의 입력 B,D가 들어감에 따라서 호환성 목적으로 active high기준으로 a~g순으로 대입시 1101101인 5가 커짐을 확인할 수 있습니다.(top모듈에서 active low로 변경)

top_bin_to_7Seg_out.vwf



BCD_to_7seg.vwf에서 active high된것을 active low로 변경 하여서 값을 나타내었습니다.

0~100ns에서 A=0,B=0이되어서 seg_tens=0,seg_ones=0

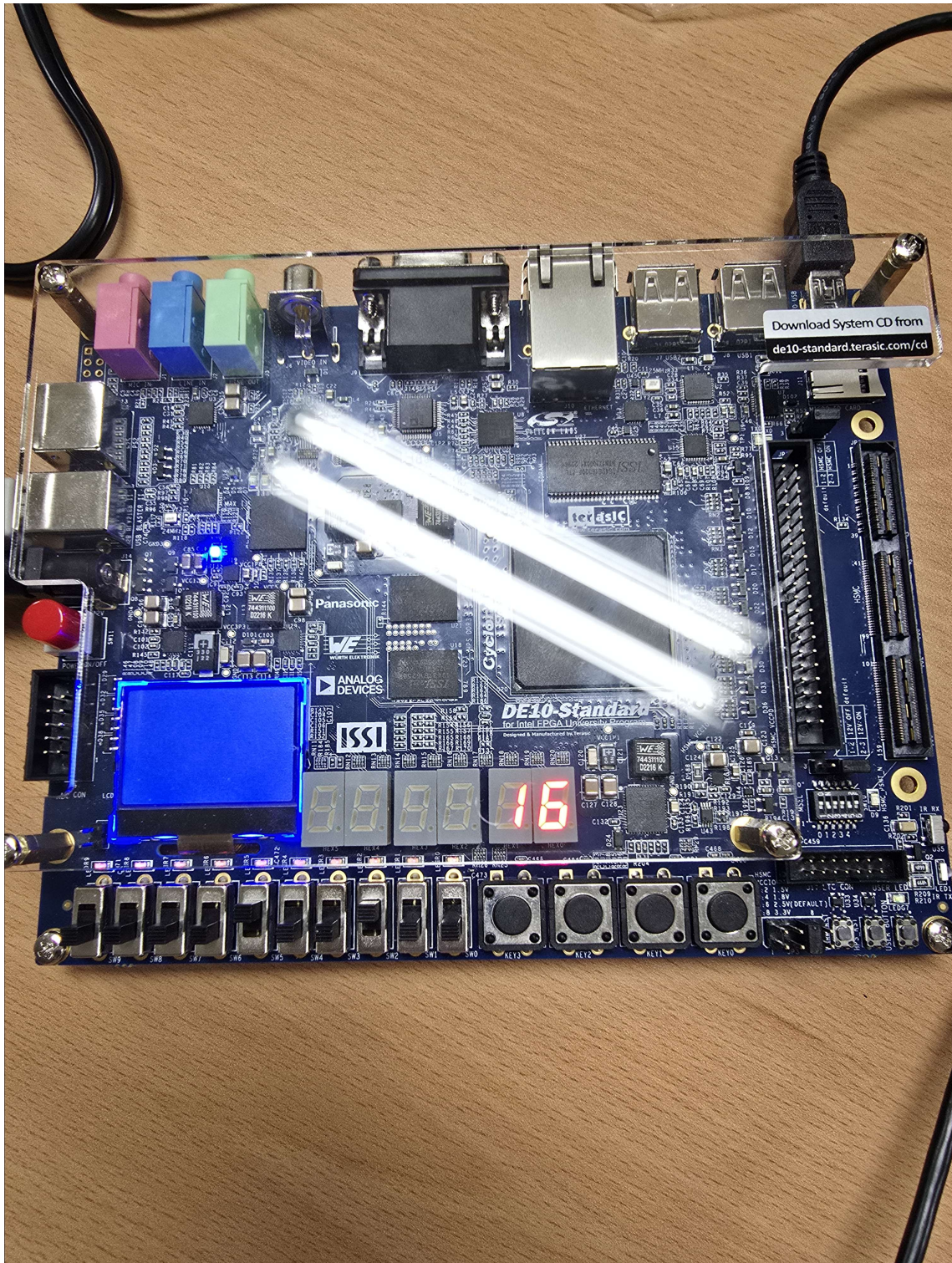
100~200ns에서 A=1,B=0이되어서 seg_tens=0,seg_ones=1

600~700ns에서 A=6,B=3이되어서 seg_tens=1,seg_ones=8

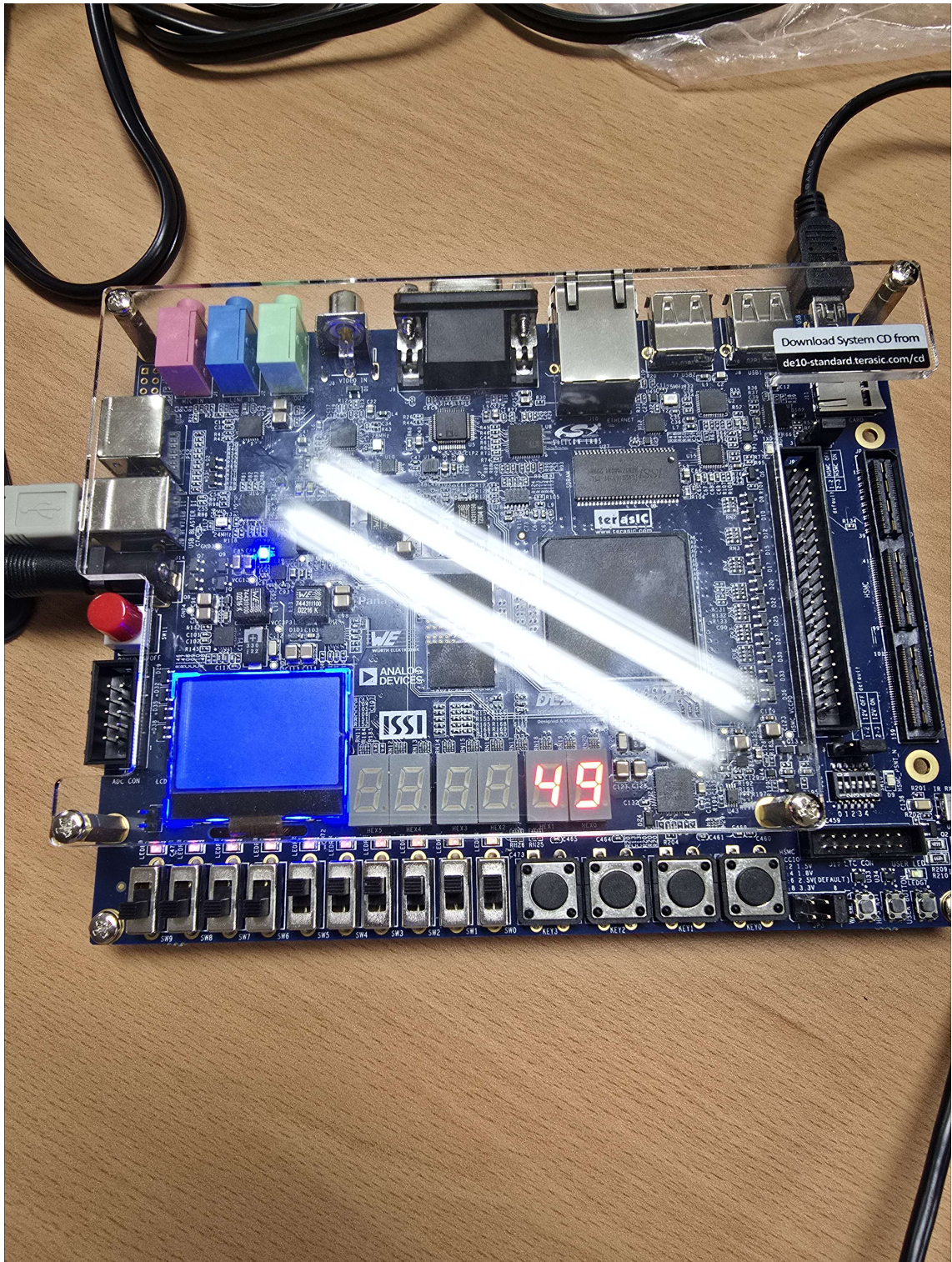
700~800ns에서 A=7,B=3이되어서 seg_tens=2,seg_ones=1

이 됨을 확인할 수가 있습니다.

6. 출력 결과



입력포트 a,b를 통해서 출력 hex0,1을 제어하였고,
 $sw5 * sw2$ 를 곱하려고 하는데 이진수 값이 모두 4 이므로,
이를 곱하면 16이 됨을 알 수가 있습니다.



입력포트 a,b를 통해서 출력 hex0,1을 제어하였고,
sw5,4,3 * sw2,1,0를 곱하려고 하는데 이진수 값이 모두
 $4+2+1=7$ 이므로, 이를 곱하면 49가 됨을 할 수가 있습니다.

7. 전체 회로도

